Check for updates

Received: 25 July 2023 | Revised: 2 February 2024 | Accepted: 27 March 2024

DOI: 10.1002/rob.22342

RESEARCH ARTICLE

WILEY

A multi-UAV system for coverage path planning applications with in-flight re-planning capabilities

Marco Andrés Luna[1,2,3] | Martin Molina[1,4] | Rodrigo Da-Silva-Gomez[1,4] | Javier Melero-Deza[1,2] | Pedro Arias-Perez[1,2] | Pascual Campoy[1,2]

[1]Computer Vision and Aerial Robotics Group at Universidad Politécnica de Madrid (CVAR-UPM), Madrid, Spain

[2]Centre for Automation and Robotics C.A.R. (UPM-CSIC), Madrid, Spain

[3]Ahyres Company, Leganes, Spain

[4]Department of Artifcial Intelligence, Universidad Politécnica de Madrid (UPM), Boadilla del Monte, Spain

Correspondence

Marco Andrés Luna, Centre for Automation and Robotics C.A.R. (UPM-CSIC), Calle Jose Gutierrez Abascal 2, 28006 Madrid, Spain.
Email: marco.lunaa@alumnos.upm.es

Funding information

European Commission; Comunidad de Madrid; Spanish Ministry of Science and Innovation

Abstract

This paper presents the development and implementation of a multiple unmanned aerial vehicle system focused on coverage path planning on multiple separated areas capable of re-planning the collective mission in case of unexpected events. For this purpose, we present a distributed-centralized architecture that uses heuristic and computationally efficient methods to perform the planning/re-planning and decision-making tasks during the control of the mission execution. We performed a computational evaluation of the algorithms, comparing them with other proposals, together with experiments in simulated and real flights. The results show that the system can distribute tasks equitably among the aircraft in an efficient way, even in the middle of the flight, when facing unexpected events; and show a higher computational efficiency when compared to multiple proposals in the state of the art.

KEYWORDS

coverage path planning, distributed systems, fault-tolerant systems, multi-UAV systems, task replanning

1 | INTRODUCTION

Coverage path planning (CPP) for unmanned aerial vehicles (UAVs) is a line of research within the trajectory planning field that has gained interest in recent years (Kumar & Kumar, 2023) with multiple applications such as aerial inspection (Shang et al., 2020), precision agriculture (Basiri et al., 2022), sterilization (Vazquez-Carmona et al., 2022), search and rescue (Cho et al., 2021), monitoring (Collins et al., 2021), and others. In multi-UAV systems, given a team of UAVs equipped with limited footprint sensors, the main task of the multi-UAV CPP approach is to find the optimal paths for each UAV to cover a polygonal area (Barrientos & Colorado, 2011).

One of the various advantages of multi-UAV systems is their fault tolerance. The robustness of such systems to uncertainties may be determined by their ability to continue the mission after damage to one or more aircraft (Hu et al., 2022) or after unforeseen changes in the state of the environment. This ability requires the development of software strategies capable of detecting unexpected events and taking control actions. In this context, a suitable control option is in-flight trajectory re-planning methods, which must be time efficient to reach the minimum cost of the mission (Luna et al., 2021; Meng et al., 2017).

On the other hand, some area coverage applications could require exploring different separated areas. For instance, post-disaster management where UAVs need to visit multiple zones; search and rescue missions with multiple candidate areas to inspect, or photovoltaic plant inspections when the panels are grouped in different zones. Based on this, Xie et al. (2018) states a new problem whose objective is to determine the optimal order of visiting each separated area and the best

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.
© 2024 The Authors. *Journal of Field Robotics* published by Wiley Periodicals LLC.

route to cover the free space to get the less cost in the mission. In multi-UAV systems, the scope of this problem extends to calculating a suitable task assignment for each agent.

Present research proposes an original computational architecture, which is applicable for the development of a multi-UAV system for coverage tasks in separate areas. This solution provides the robotic system with the feature of in-flight automatic unexpected event detection and route re-planning, ensuring mission continuity and equitable task distribution among the remaining UAVs. The architecture is described as an interconnected set of information processes with their corresponding algorithms. The proposed solution follows a hybrid centralized-distributed approach including both global planning for multiple vehicles and local planning for each individual UAV.

The rest of the paper is organized as follows: Section 2 presents a literature review, and Section 3 states the problem definition. Next, Section 4 outlines the system architecture including the different information processes and their algorithms. Then, in Section 5, simulation and real results are reported. Finally, in Section 6 conclusions and future work are described.

2 | RELATED WORK

In the literature, there are multiple proposals to solve the multi-UAV CPP problem. The current approaches can be divided into three-dimensional (3D) and 2D CPP. In this context, 3D-CPP applications include visual inspection, terrain following, and others. For example, the research presented by Melo et al. (2021) uses linear optimization and heuristic methods to generate waypoints in the 3D space for visual inspection. Others like Mansouri et al. (2018) slice the 3D space into 2D and generate coverage waypoints at different altitudes. For their part, proposals in Li et al. (2018), Mou et al. (2021) employ genetic algorithms and deep reinforcement learning, respectively, for a 3D terrain exploration. For the 2D case, the survey presented by Cabreira et al. (2019) distinguishes between decomposition and no-decomposition methods. Decomposition techniques divide the coverage area into sub-areas to be explored by simple UAV motions; examples of this method are found in Choi et al. (2020), Maza and Ollero (2007), Nedjati et al. (2016), Valente et al. (2013) where the authors divide the area into cells based on the sensor footprint and found the best route to visit all the cells using optimization techniques. Non-decomposition methods use geometric patterns to cover the areas as back-and-forth (Avellar et al., 2015; Hong et al., 2021; Luna et al., 2022) or spiral pattern (Balampanis et al., 2017; Luna et al., 2023; Semsch et al., 2009; Zhu et al., 2021); in this type of algorithm, the authors use different techniques to find optimal routes and optimal task distribution for each UAV. In (Torres et al., 2016), the authors combine different methods using a geometric back-and-forth pattern in convex polygonal areas; but cellular decomposition in concave regions.

Furthermore, several methods in the literature have dealt with CPP in multiple areas. For instance, Xie et al. present through their research (Xie & Chen, 2022) different approaches to solve the problem that includes dynamic programming (DP), nearest neighbor (NN), genetic algorithms (GA), and heuristic methods. They applied these techniques to find the optimal coverage route for all the regions using back-and-forth patterns with numerical simulations for single and multiple UAVs. On the other hand, the authors in Vasquez-Gomez et al. (2018) decompose the problem into route generation and visiting area order optimization and test their algorithms in a single UAV system. In addition, the research presented in Yu et al. (2020), describes a multi-regional solution for multi-UAVs based on the ant colony optimization technique to obtain optimal results; they assume the same launch point for all aircraft. Similarly, Khanam et al. (2020) proposes mixed integer linear programming and heuristic techniques for inter-region and intra-region path planning considering the precedence provision in visiting regions. These approaches were evaluated through numerical simulations considering that the aircraft has sufficient energy capacity to complete the entire route. Finally, the research presented in Flores Peña et al. (2022) addresses this problem using a near optimal TSP-based solution for the multiple-area problem. Most of the works in the literature address the problem in simulation due to its implementation complexity. On the other hand, the main challenge faced by this type of algorithm is the computational complexity when it is required to optimize the total distance traveled in several separate areas (specifically, more than 20 areas).

However, path planning represents only the first part of the development; the implemented techniques must consider that aircraft are subject to all types of failures in the real world, and this issue becomes more complex in UAV swarms. For this reason, several works in the literature have studied the problem of fault-tolerant swarm systems (Fourlas & Karras, 2021; Ziquan et al., 2022). In the research presented by De Benedetti et al. (2017), Hu et al. (2022), Tahir et al. (2020), the authors address the problem of fault-tolerant formation control in UAV swarms. The main idea is to maintain a regular geometric formation even if one or more aircraft fail in flight. Others like Wang et al. (2021) propose an optimal task management model with distributed task reassignment in case of UAV failure. In addition, further investigations explore the mission replanning recourse. For instance, works like Chen et al. (2022), Meng et al. (2009) study the task replanning problem in dynamic environments, recalculating the optimal path in certain events, these investigations determine the shortest route for exploring multiple waypoints and replanning in case of finding new information, but they do not address the area coverage problem. Moreover, the research presented in Hong et al. (2022) address the CPP problem with in-flight replanning using a heuristic algorithm called RHTA. In this architecture, two types of replanning events are handled. First, when the operator adds a new UAV to the system, and second, when a new region to explore is added. The authors do not propose automatic event detection heuristics. Additionally, no further research related to our proposal has been found.

3 | PROBLEM DEFINITION

In this research, we assumed a group of N rotary-wing UAVs located at different positions with the mission of covering M convex polygonal separated areas defined by an operator as multiple sets

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

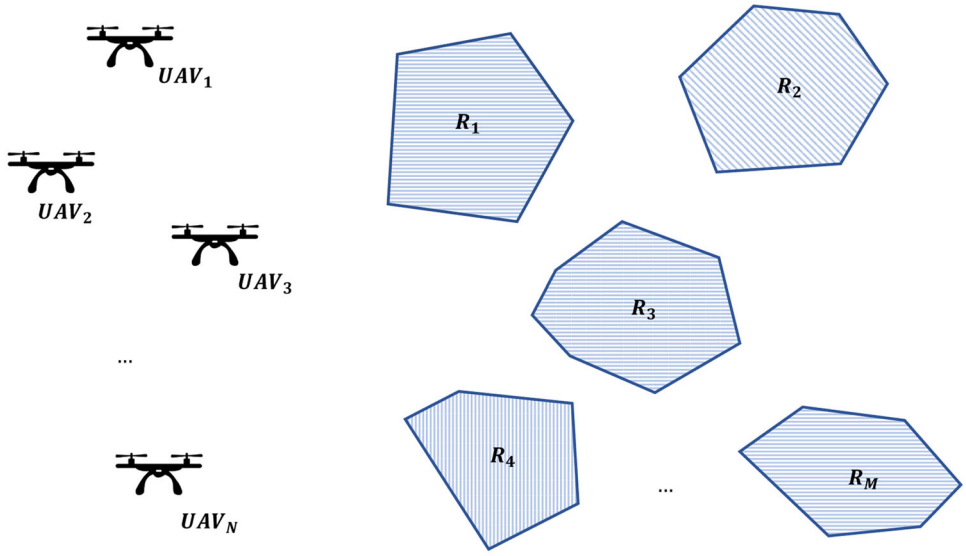


FIGURE 1 Mission operation schema.

of P_n points in the IR^2 space, as shown in Figure 1. Considering the practical applications of the problem, the number of disjoint areas has been limited to a maximum of 10.

Additionally, the group of UAVs is homogeneous; therefore, their structure and equipment for aerial sensing tasks are the same. Also, the aircraft has four degrees of freedom, but the turning cost is minimal, considering that the sweep is performed with a fixed heading. For this reason, the altitude during the coverage flight will depend of sensor footprint, but it will be the same for all the UAVs. Thus, this footprint will also determine the the width of the lanes, and the spacing between waypoints (as shown in Figure 2).

Besides, all UAVs will take off at the same time. For safety reasons, each UAV will climb and navigate to the operation zone at different altitudes, and once there, they will descend to the mission altitude for the coverage task (as shown in Figure 3).

Finally, during the operation, the system must perform a fast re-planning during the mission if one or more aircraft experience an erroneous behavior such as return to launch point, hover, landing, wrong navigation, and others. At this point, it should be noted that each waypoint should not be visited more than once.

For this proposal, we use an extended version of the well-known back-and-forth method since it is a common technique implemented by several works in the literature (Avellar et al., 2015; Hong et al., 2021; Xie & Chen, 2022); also, it is the default used technique in multiple ground control station software such as QGroundControl,[1] Mission Planner,[2] UgCS,[3] and others.

On the other hand, we assume an obstacle-free environment considering that UAVs are not equipped with sensors to detect them;

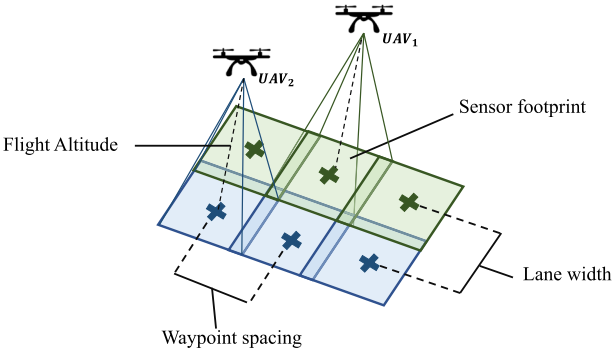


FIGURE 2 Sensor footprint schema.

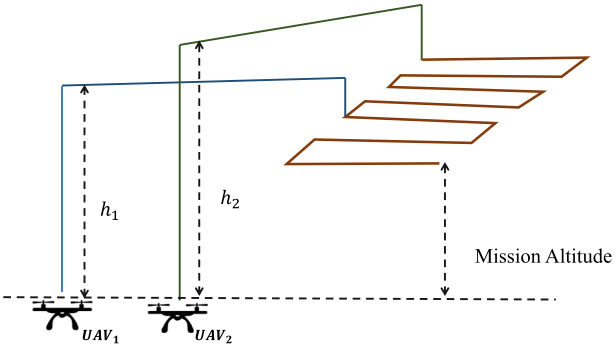


FIGURE 3 Flight altitude schema.

in addition, this type of environment is common in several practical outdoor applications such as photovoltaic inspections, precision agriculture, or mapping, in which UAVs fly high enough to avoid any obstacles. However, the architecture developed in this research can

[1]http://qgroundcontrol.com/
[2]https://ardupilot.org/planner/
[3]https://shop.ugcs.com/

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

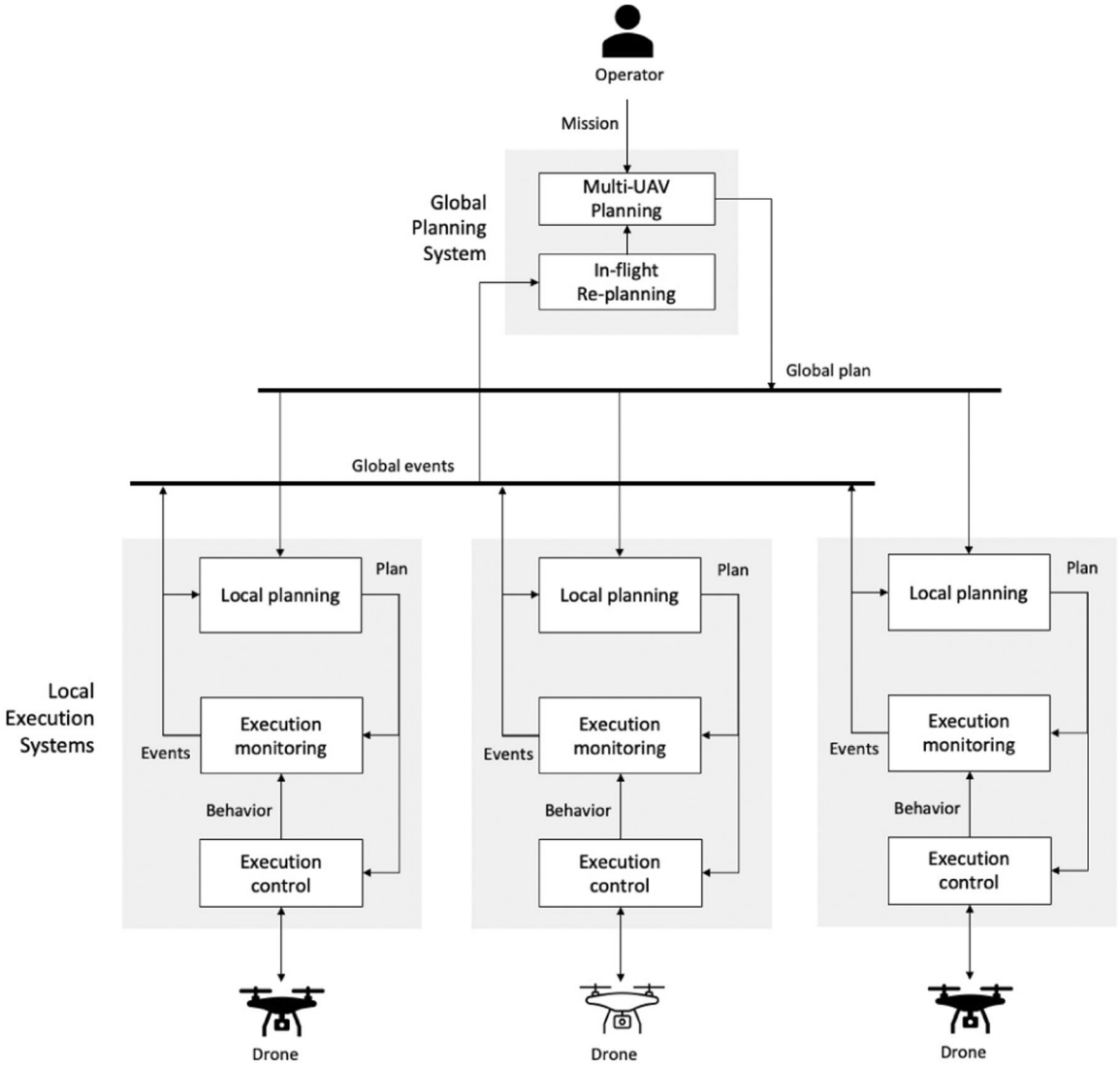


FIGURE 4 Proposed control architecture for multi-UAV systems. In this figure, each rectangle represents an information process and each arrow corresponds to specific data that flows from one process to another one.

be used with improved planning and replanning algorithms capable of optimizing area coverage in more complex environments with obstacles.

Alternatively, to above mentioned methods for single UAV area coverage, UAV software architecture for multiple aircraft have captured the attention of the UAV swarm intelligence scientists (Zhou et al., 2020). For instance, the approaches of Molina et al. (2019), Sanchez-Lopez et al. (2016) propose a Robotic Operating System (ROS) centralized software architecture to ensure a concurrent aerial mission execution with multiple UAVs. The authors in Pinto et al. (2021) present a centralized cognitive architecture that integrates different methods (including those for UAV coordination) for a multi-UAV decision-making system, and the research in (Emel'yanov et al., 2016) states a multilayered control architecture to manage UAV swarm tasks.

4 | PROPOSED CONTROL ARCHITECTURE

4.1 | General structure

Figure 4 shows the control architecture that we propose in this paper for multi-UAV systems. The architecture follows a hybrid centralized-distributed approach. On the one hand, there is a *global* planning system that generates a complete plan for all UAVs and it is able to modify dynamically the plan when certain events happen. On the other hand, there is a *local*execution system for each particular UAV which, besides to be able to execute the plan, it is able to monitor its execution and adapt it locally when certain events happen.

This control architecture includes a number of processes that operate as follows. The global planning system initially generates a global plan using as input the mission goals provided by the human

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

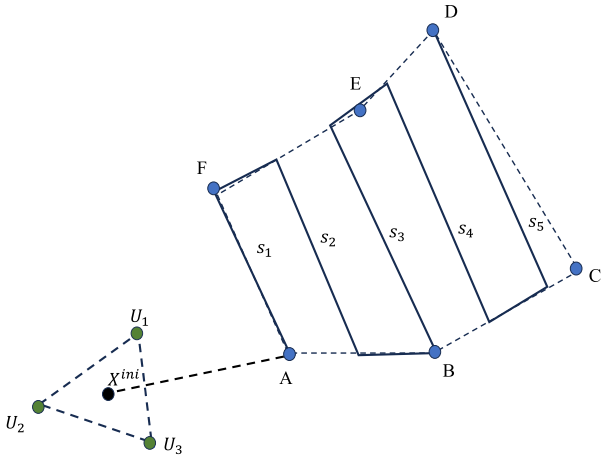


FIGURE 5 Back-and-forth schema. U_1, U_2 and U_3 are the initial UAV locations. The B&F technique generates and joins multiple segments (s_1, s_2, ..., s_5) through the polygon *ABCDEF* according the optimal sweep direction criteria (Huang, 2001) and sensor footprint parameters.

operator. This is done by the process called *multi-UAV planning*. The generated global plan is received by each local execution system which selects and refines locally the part of the global plan that corresponds to its specific UAV. This is done by the *local planning* process. Then, the *execution control* process executes the plan by sending the adequate control commands to the aerial platform. During the execution of the plan, there is a local *execution monitoring*process for each UAV that operates in parallel observing the behavior of the UAV to detect the presence of relevant events such as unexpected situations that were not considered in the initial planning. When an event is detected, it may be handled locally (through the *local planning* process) or globally (through the *in-flight re-planning* process) and, then, a new plan (local or global) is generated to cope with the new situation. Next sections describe in more detail the procedures followed by these processes.

4.2 | Multi-UAV planning

In our previous research (Luna et al., 2022), we used the back-and-forth strategy (B&F) (Huang, 2001) to generate the coverage routes. This method uses the vertex information and the parameters of the UAV sensor footprint to define the sweep direction and the width of the lanes. As shown in Figure 5, our method calculates the paths for one region, taking as reference the centroid x^{ini} of the polygon formed by the initial positions of the UAVs.

For the present approach, the planning technique was extended to cover multiple separated areas. Thus, we propose a near-optimal heuristic algorithm that receives a set of convex regions to generate B&F routes for each polygon to find the sequence of regions to be visited iteratively, taking as a reference the shortest distances between the starting and end points of each coverage route. (as observed in Figure 6).

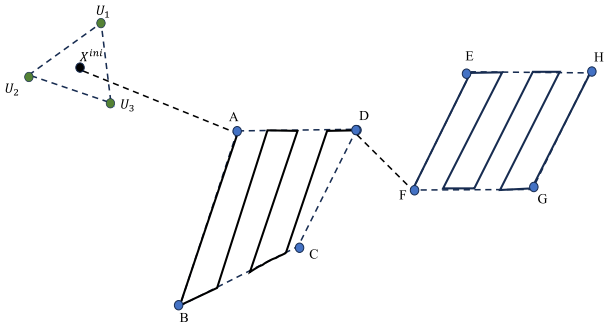


FIGURE 6 Extended planning schema. U_1, U_2, U_3 are the initial UAV locations. B&F generate the route the polygon *ABCD* starting from A and finishing in D taking X^{ini} as initial reference point. Applying the extended planning algorithm, the point *D* will be the reference of the next polygon and *F* the start of *EFGH* polygon coverage route.

Figure 6 shows a diagram of the method for two areas. To determine the first region to cover, the centroid of the polygon generated by the initial positions of the UAVs is used as a reference for which *A* is the starting point, and *ABCD* is the first polygon. Then, region *EFGH* takes *F* as the initial point following the shortest distance criteria. Algorithm 1 describes the process formally for multiple regions and multiple UAVs.

Algorithm 1: Multi-area Coverage

The Data: U, R, lw
The Output: W

```
1  W = null
2  x^ini ← Centroid(U)
3  B ← B&F(R, lw)
4  B^ini ← FirstPoints(B)
5  while B ≠ null do
6  |   B^min = MinDist(x^ini, B, B^ini)
7  |   W ← Append(B^min)
8  |   B ← Remove(B^min)
9  |   x^ini ← GetLastPoint(B^min)
10 end
```

Algorithm 1 takes as inputs the vector of initial positions of the UAVs U (a list of points), the set of polygons R (each polygon represented with a list of points), and the lane width lw. The result will be a vector of waypoints W. First, we initialize the empty vector W. Then, we obtain the initial point of the route x^{ini} by calculating the centroid of the positions vector U. Then, we apply the $B\&F()$ function to the polygon vector to store the resulting routes in B, one route per polygon. In addition, the *FirstPoints*() function calculates the vector of the first points of each path in B, storing them in the vector B^{ini}. Then, the method starts an iterative process for each path B, calculating B^{min} as the route with the closest starting point to x^{ini} with the *MinDist*() function. At this point, the points of B^{min} route are added to the vector of waypoints W. Then, B^{min} is removed from the B vector, and the new x^{ini} is calculated by getting the last point of B^{min}

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

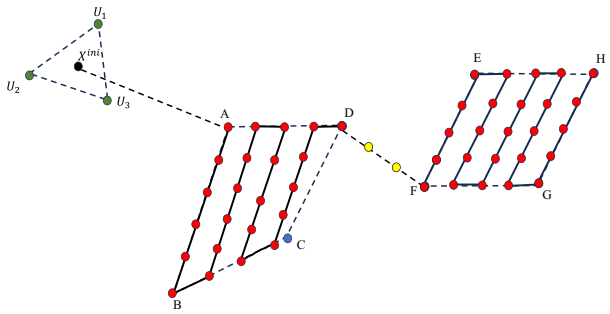


FIGURE 7 Sampled B&F schema. Red points are labeled as *inspection points* and yellow points are labeled as *route waypoints*.

with the function *GetLastPoint*(). The algorithm repeats the process until all the routes are added to W.

This computationally efficient heuristic solution is a near-optimal alternative for the multiple areas visiting order problem, but unlike other works in the literature Flores Peña et al. (2022), Vasquez-Gomez et al. (2018), Xie and Chen (2022), this technique does not globally optimize the total distance traveled; which means that the mission cost can increase if the number of regions increases significantly. However, as shown in the results in Section 5, the computation time is reduced. Therefore, we use this method to prioritize planning time, given that in real-world applications such as photovoltaic plants or search and rescue operations, the number of explored areas will be small (between 2 and 10).

Finally, unlike other works in the literature Avellar et al. (2015), Hong et al. (2021), the B&F route lines are divided into multiple points considering the value of a prefixed parameter that defines a constant distance between consecutive waypoints. These waypoints are then labeled as *inspection points* if they are part of the coverage route or *route waypoints* if they are part of the route between areas. This final division and labeling simplify the task distribution among the agents and the re-planning process (Figure 7).

For the multi-UAV task distribution we use the Powell-BINPAT algorithm (Luna et al., 2022) that addresses the CPP problem as a weighted task optimization problem (Gerkey and Matarić, 2004). The objective of this technique is to minimize the total mission time. Its main advantage is its low computational cost. In this context, considering the set of waypoints W obtained in Algorithm 1, and assuming a constant flight speed, Equation (1) represents the individual cost of the mission for each UAV.

$$C_m = \sum_{i=1}^{k-1} \left\| X_i - X_{i+1} \right\| - T_m + Hd_m. \tag{1}$$

In Equation (1), m is the number of UAVs, and k is the number of waypoints in the global plan. The individual cost of each UAV C_m is the Euclidean distance between two consecutive waypoints $\|X_i - X_{i+1}\|$, T_m is a binary function that determines which subset of routes must be assigned to the UAV and Hd_m is the delay caused by the differences of altitudes and positions. Thus, the task distribution algorithm must determine the subset of waypoints corresponding to each UAV that optimizes the overall cost of the mission. For this, the BINPAT approach is presented in Algorithm 2.

Algorithm 2: BINPAT Algorithm

The Data: U, W
The Output: M
$CVec \leftarrow null$
$W' \leftarrow null$
$GlobalCost \leftarrow GetCost(W)$
$Weights \leftarrow GetWeights(U)$
$C^{max} \leftarrow GlobalCost \cdot Weights$
foreach $U_i \in U$ **do**
  $Cost \leftarrow 0$
  while $Cost < C^{max}$ **do**
    $W' \leftarrow GrupPoints(W)$
    $Cost \leftarrow GetNewCost(W'_i)$
  end
  $CVec \leftarrow Append(Cost)$
end
$Assignments \leftarrow JV(W', U)$
$M \leftarrow SetAltitudes(Assignments, CVec)$

As mentioned before, Algorithm 2 takes the set of waypoints (W) obtained in the previous stage and the UAV positions (U) as input data to produce the mission plan (M). First, we initialize the cost vector *CVec* and W' as a list that contains the waypoints grouped by each UAV. Then, we get the global cost of the mission with the function *GetCost*() that sums the total distances between all the individual waypoints. In the next stage, the *GetWeights*() function obtains the weighted value of the UAVs so that $0 \leq weight \leq 1$.

As it is a homogeneous system, this value will be the same for all of them and depends on the number of agents; for instance, in five aircraft, this value will be 0.2 (or 1/5), and for 4 UAVs equal to 0.25 (or 1/4), and so forth.

Next, the maximum cost vector (C^{max}) is the result from the dot product between *Weights* vector and the global mission cost (*GlobalCost*). As this method considers each UAV as a weighted package (according to the analogy with the bin packing problem), the waypoints for each drone are grouped in W' with the *GroupPoints*() function until the distance cost (*Cost*) of each group of points is less than the maximum cost for each UAV (C_i^{max}). Considering the different initial positions of the problem, we perform an assignment task using the Jonker-Volgenant algorithm (*JV*() function) (Jonker & Volgenant, 1987) to assign the plans based on the reach and return plan distances for each UAV. Finally, the function *SetAltitudes*() calculates the initial altitude for each UAV, considering the difference in altitude necessary to avoid collisions until reaching the initial point of the mission. Then, each UAV reaches the inspection altitude, and at the end of the coverage mission, they resume their altitude to avoid collisions

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

while returning to the takeoff point. The output of this function is the final set of waypoints for each UAV.

However, due to the difference in the initial position of the UAVs and altitudes mentioned before, this distribution is still not optimal. For this reason, as additional step, the initial weights subject to constraints in Equation (2) are optimized using Powell's method (Powell, 1964) to minimize the maximum individual mission cost (Equation 3). The Powell method performs an iterative process to find the set of weights W_k by bidirectional line search along each search vector in the N-dimensional set of solutions:

$$0 \leq W_m \leq 1, \forall\, m \in M, \tag{2}$$

$$\min(\max C^m). \tag{3}$$

4.3 | Execution monitoring

The objective of execution monitoring is to observe how each drone executes the inspection plan and to report when the inspection task has been completed successfully or when an unexpected situation happens. Unexpected situations correspond to changes in the assumptions considered during the planning. These assumptions include speed decreasing during the flight, maximum lifetime of the battery, appropriate distance to the target to take images, a new UAV added to the system, etc. During the execution, these assumptions may differ due to changes in the environment or UAV conditions (presence of high wind, obstacles, UAV failures, etc.).

The monitoring task is performed in a distributed way, that is, there is a monitoring process for each UAV which operates continuously during the execution of the plan. For this, the execution monitoring process receives as input (1) the flight plan represented as a sequence of robot behaviors (e.g., go to a certain point, take-off, land, etc.) and (2) periodic messages about the completion of robot behaviors and about the state of the vehicle (e.g., battery charge, camera status, etc.).

Table 1 shows examples of events that may be generated during this process. For instance, the *UAV lost* category corresponds to a situation when the UAV is following a path that differs significantly from the planned path or when the camera is not able to take images because it is out of order. The category called *poor image acquisition* represents that the UAV does not acquire a correct image through the cameras. This issue could be the result of shadows or blurs in the data. The *UAV available* category represents that a UAV is added to the system and available to execute a new plan. This happens when a UAV has completed the inspection task (but it has not yet returned the home base) or when the operator has added a new UAV to the system, or when UAV that had been lost becomes available again. Other categories are battery decay (the UAV has an unexpectedly huge battery decay) or manual control (the human operator takes the manual control of the UAV).

The monitoring process operates in a continuous loop. In each cycle, the monitoring process analyzes the presence of potential events in a prefixed order of priority and stops the monitoring cycle when it detects an event. In this way, in the presence of multiple events, the highest priority event is handled first and the rest of events are processed in the next monitoring cycles.

Events can be detected by the monitoring process in two different ways. On the one hand, some events (e.g., deviations from the planned path or battery charge decay) are detected by performing certain local operations (see more details below). On the other hand, certain events are detected directly by receiving specific messages from other processes. These messages correspond for example to the activation of manual control or to the presence of a new available UAV. In this paper, we assume that the camera image quality is notified by a message that is generated by an external process (e.g., manually by the operator or automatically by a computer vision algorithm).

To verify the correct flight motion, the execution monitor keeps track of the visited and to-be-visited waypoints (this information is extracted from the completed behaviors of the flight plan). We assume that a point has been visited correctly when the UAV has flown over it, and in the case of inspection points, the images acquired through the camera are of sufficient quality.

To check if a UAV is flying correctly over a planned point, we define γ as the distance threshold in meters that a UAV must deviate from its trajectory to be detected as lost. To evaluate this condition we calculate the distance to the trajectory (D_{tr}) as the distance from the point ($P_{current}$) to the straight line formed by the last and the next waypoints ($\overline{P_{last}P_{next}}$) given by the following equation:

$$D_{tr} = \frac{(x_{next} - x_{last})(y_{last} - y_{current}) - (x_{last} - x_{current})(y_{next} - y_{last})}{\sqrt{(x_{next} - x_{last})^2 + (y_{next} - y_{last})^2}}. \tag{4}$$

TABLE 1 Examples of events that may be detected by the monitoring process.

Event	Description
UAV lost	The UAV is following a path that differs significantly from the planned path or the camera is not able to take images because it is out of order
Poor image acquisition	Camera image is blurred or has shadows that prevent a correct inspection
Battery decay	The battery of the UAV has a unexpected huge battery decay
Manual control	The human operator has taken the manual control of the UAV
UAV available	A UAV has completed the inspection mission, a UAV that was lost is now available or a new available UAV is ready to participate in the mission

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

In Equation (4), $x_{current}$ and $y_{current}$ are the coordinates of the 2D projection of the UAV ($P_{current}$), $P_{last}(x_{last}, y_{last})$ is the previous visited waypoint and $P_{next}(x_{next}, y_{next})$ is the upcoming waypoint. This condition will be detected by the function $dev(P_{current}, Plast, Pnext)$ (Equation 5) that detects when the UAV has deviated sufficiently from its trajectory to be considered lost.

$$dev(P_{current}, P_{last}, P_{next}) = \begin{cases} 1, & \text{if } D_{tr} > \gamma \quad \text{Equation (4)}, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

To determine if the UAV is navigating in the correct direction, its distance to the next waypoint is constantly calculated. Given *dcurr* defined as the distance to the current waypoint defined by the following equation:

$$d_{curr} = \|P_{next} - P_{current}\|. \tag{6}$$

And P_{prev} the position in the previous instant, the distance from the UAV to the next waypoint (d_{prev}) is defined by the following equation:

$$d_{prev} = \|P_{next} - P_{prev}\|. \tag{7}$$

If the distance to the next waypoint increases rather than decreases, and this increase is greater than the γ threshold, the system reports that the UAV is lost (Equation 8).

$$ret(d_{curr}, d_{prev}) = \begin{cases} 1, & \text{if } (d_{curr} > d_{prev}) \wedge \|d_{curr} - d_{prev}\| > \gamma, \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

Finally, we define δ as the distance threshold that determines when a waypoint is visited. The function *LblVis* evaluate this condition (Equation 9)

$$LblVis(P_{current}, P_{last}) = \begin{cases} 1, & \text{if } \|P_{current} - P_{last}\| > \delta, \\ 0, & \text{otherwise.} \end{cases} \tag{9}$$

In summary, the γ threshold is used to determine if a UAV is tagged as lost and δ to monitor if a waypoint has been visited, these values are not fixed and must be adjusted based on the operating range of the flight plan.

4.4 | Local planning

The objective of local planning is to generate a plan specified in detail for a particular vehicle. This task is performed in a distributed way, that is, there is a local planning process for each UAV. One of the goals of this process is to extract the part of the global plan corresponding to the specific vehicle and translate it from a representation expressed as a list of waypoints to a more detailed representation in form of a sequence of robot behaviors. Figure 8 shows a partial example of a flight plan which is formulated as a sequence of robot behaviors (take off, go to and land).

```
"plan": [
    {"takeoff":   {"height": 7.0, "speed": 1.5}},
    {"go_to":     {"x": 3.5, "y": 0.0, "z": 7.0, "speed": 3.0,
                   "yaw_mode": KEEP_YAW}},
    {"go_to":     {"x": 3.5, "y": 5.0, "z": 7.0, "speed": 3.0,
                   "yaw_mode": KEEP_YAW}},
    {"go_to":     {"x": 0.0, "y": 5.0, "z": 7.0, "speed": 3.0,
                   "yaw_mode": KEEP_YAW}},
    {"land":      {"speed": 0.5}}
]
```

FIGURE 8 Partial example of a mission plan for an aerial vehicle. The plan is expressed as a sequence of robot behaviors.

Another goal of local planning is to generate new local actions (which were not specified in the global plan) as a response of certain events. Table 2 shows examples of local actions generated in this way. For each event, there are actions that are generated by the local planner of the affected UAV and other actions generated by the local planners of the rest of UAVs. For example, if the event battery decay is detected, the local planner of the affected UAV generates actions to land immediately in a safe area, while the local planners of the rest of UAVs generate the action of waiting for a new global plan (see more details below). If the event poor image is detected, the local planner of the affected UAV repeats the inspection of the last points and no actions are generated for the rest of UAVs.

Waiting for a new global plan is an important action to be done locally, to adequately synchronize the re-planning process. This action performs the following procedure. First, the vehicle v stops the execution of its current local plan p, stopping the motion flight during a brief time Δt (e.g., $\Delta t = 2$s). This is done to give time to the global planner to generate a new plan (which includes asking for specific data to all vehicles and taking the time to search for the new plan). During this time, it is important not to change the spatial location of all vehicles assumed by the global planner. Then, if the global planner generates a new plan p' for the vehicle v, the vehicle starts the execution of p'. Otherwise, the vehicle v continues the execution of the previous plan p.

4.5 | In-flight re-planning

Re-planning is a decisive task to enable the capability of in-flight adaptation of the entire system to situations where there are changes concerning the initial conditions in which the algorithm generated the plan. Whenever the execution monitoring process detects a significant event, the global planner performs the re-planning method to cover the remaining waypoints with the lowest mission cost. For this purpose, we use as inputs the waypoints defined as *inspection points* and *route waypoints* defined in Section 4.2 together with the waypoints labeled as *visited*, and the events of the execution monitoring described in section 4.3. In this context, while inspection

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 Examples of actions performed in local planning.

Event	Local actions for the affected UAV	Local actions for the rest of UAVs
UAV lost	The UAV is commanded to return to the home base	Wait for new plan
Poor image acquisition	The UAV repeats the last n points	Continue with current plan
Battery decay	The UAV is commanded to land immediately in an safe area	Wait for new plan
Manual control	The local plan for this UAV is deleted	Wait for new plan
UAV available	Wait for new plan	Wait for new plan

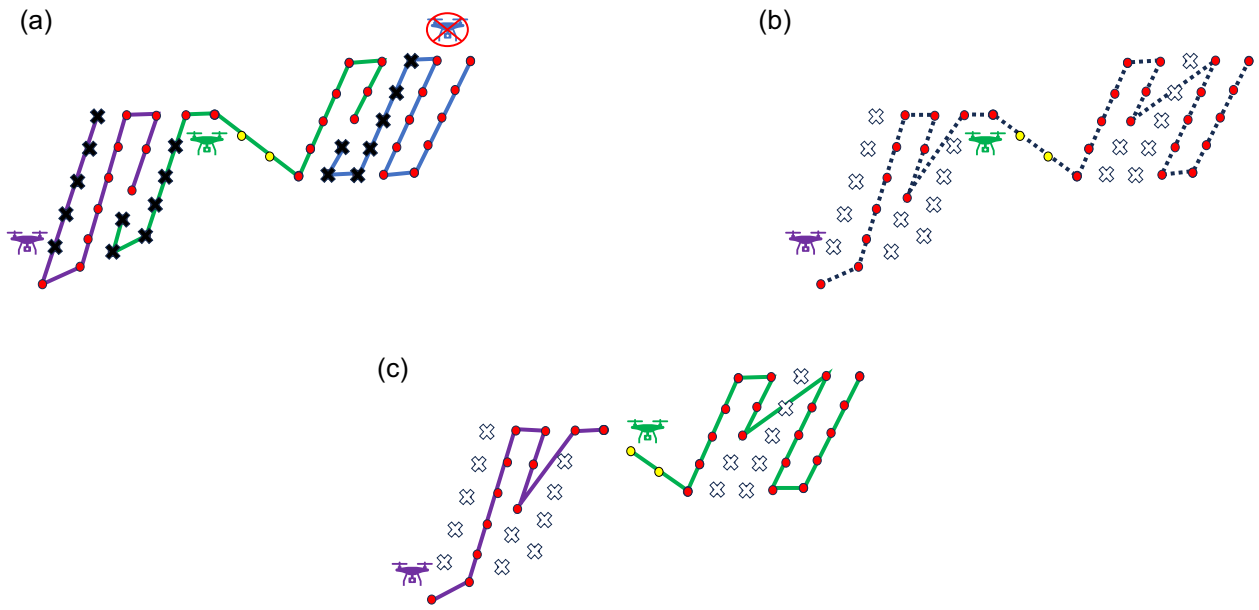


FIGURE 9 Replanning scheme: (a) Multi-UAV plan. Powell-BINPAT assign one route per UAV (purple, green, blue); visited waypoints are marked with "x," and blue UAV is detected as *lost*. (b) Unvisited waypoints concatenation. (c) BINPAT distribution. Purple and green routes are generated and new missions are reassigned for the remaining drones.

waypoints are located within the areas to be covered, the positioning waypoints are only used to support the navigation of the UAV.

During the re-planning phase, the planner concatenates the unvisited inspection waypoints to produce a different route generating new inspection and position waypoints (as shown in Figure 9). Figure 9a shows the execution of the initial plan by three UAVs (purple, green, and blue). The extended *B* & *F* technique produces the global route; then Powell-BINPAT distributes it among the UAVs. During the navigation, the waypoints are tagged as visited using the label condition with δ threshold mentioned in Section 4.3. And at a given moment, the system detects that the blue UAV is lost. In Figure 9b, a new global path is generated by concatenating the unvisited waypoints; in this stage, the algorithm places new *route waypoints* to support the navigation. Finally, in Figure 9c, the algorithm distributes the task over the new path using the Powell-BINPAT approach again, and the mission continues on the fly.

To ensure the efficiency of the planner, the system calculates the cost of the new mission. In cases where the algorithm has to replan because one of the UAVs has completed its task (i.e., has visited all its points), this cost is used to determine whether to perform this replanning. Thus, let W^O be a sequence of trajectories, the weighted cost $C\omega$ of the trajectory is defined as:

$$C\omega(W^O) = \max(lenght(W_i));\ 1 \le i \le n. \tag{10}$$

That is the length of the longest path in the sequence. With the weighted cost and given a defined threshold ϕ, a new sequence W^N will be executed only if:

$$\frac{C\omega(W^O)}{C\omega(W^N)} \le \phi. \tag{11}$$

4.6 | Execution control

The objective of the execution control process is to execute the plan assigned to the UAV. This process interprets the flight plan (expressed a a sequence of robot behaviors) to generate the appropriate control commands to the aerial platform. In addition, the execution control process reports periodically the state of the execution (e.g., completion of the behavior, position and orientation of the robot, etc.), which is useful for the execution monitoring process.

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

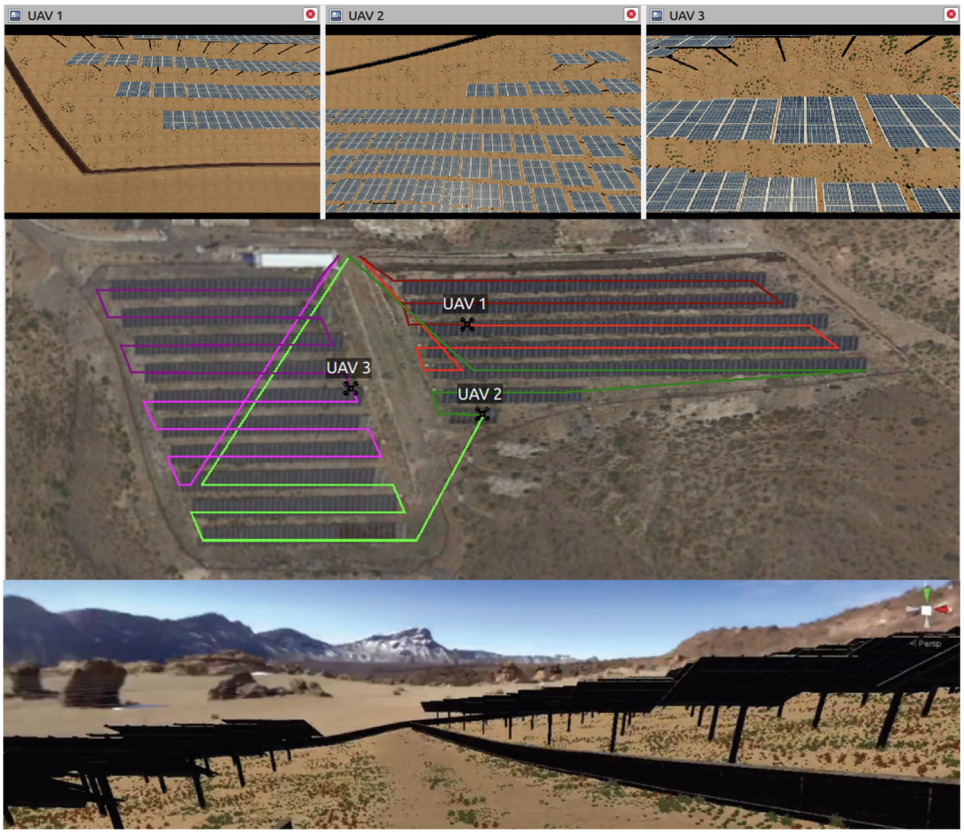


FIGURE 10 Example images generated during the experiments performed for multiple UAVs using the Flightmare simulator.

In our architecture, we have implemented the execution control using the Aerostack2 software framework (Fernandez-Cortizas et al., 2023). Aerostack2 is specialized in aerial robotics and is applicable to different types of platforms, both real (e.g., DJI Matrice, Crazyflie, etc.) and simulated (e.g., using Gazebo simulator). Aerostack2 has been used for missions with multi-robot systems and is based on Robot Operating System 2 (ROS2) (Macenski et al., 2022).

Aerostack2 includes a specialized software component called *mission plan interpreter* that facilitates the execution of plans. This interpreter allows an incremental adaptation of mission plans during their execution, which is very appropriate to support the re-planning capability presented in this paper. This means that during the execution of a specific plan p, the interpreter may receive as input an order (together with an optional plan p') that modifies the execution of p. Examples of orders are: *insert* p' (execute p' and, then, the rest of p), *append* p' (execute p' at the end of the execution of p), *pause* (pause the execution of p), etc.

During the execution of a flight plan, the mission plan interpreter of Aerostack2 publishes periodically data about how the sequence of robot behaviors is being executed. This data includes: (1) the behavior that is currently running, (2) feedback from the behavior execution (which is particular for each behavior), and (3) the number of behaviors completed and pending in the plan. For example, given the plan described in Figure 8, at some point of the execution, the data reported by the mission plan interpreter could be: (1) the behavior currently executing is *go_to*, (2) the status of the behavior *RUNNING*, (3) the specific feedback is (*actual _speed* = 2.983, *distance_to_goal* = 4.571), and (4) there is one behavior completed and three pending.

TABLE 3 Mission cost and coefficient of variation (CV) results for different number of UAVs and three exploration areas.

UAVs	Mission cost (m)	CV
2	674.47	0.59
3	493.95	4.75
4	374.19	4.35
5	319.92	5.63
6	286.31	15.59
7	250.11	8.24
8	226.32	8.11
9	206.91	6.08
10	193.16	9.05

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

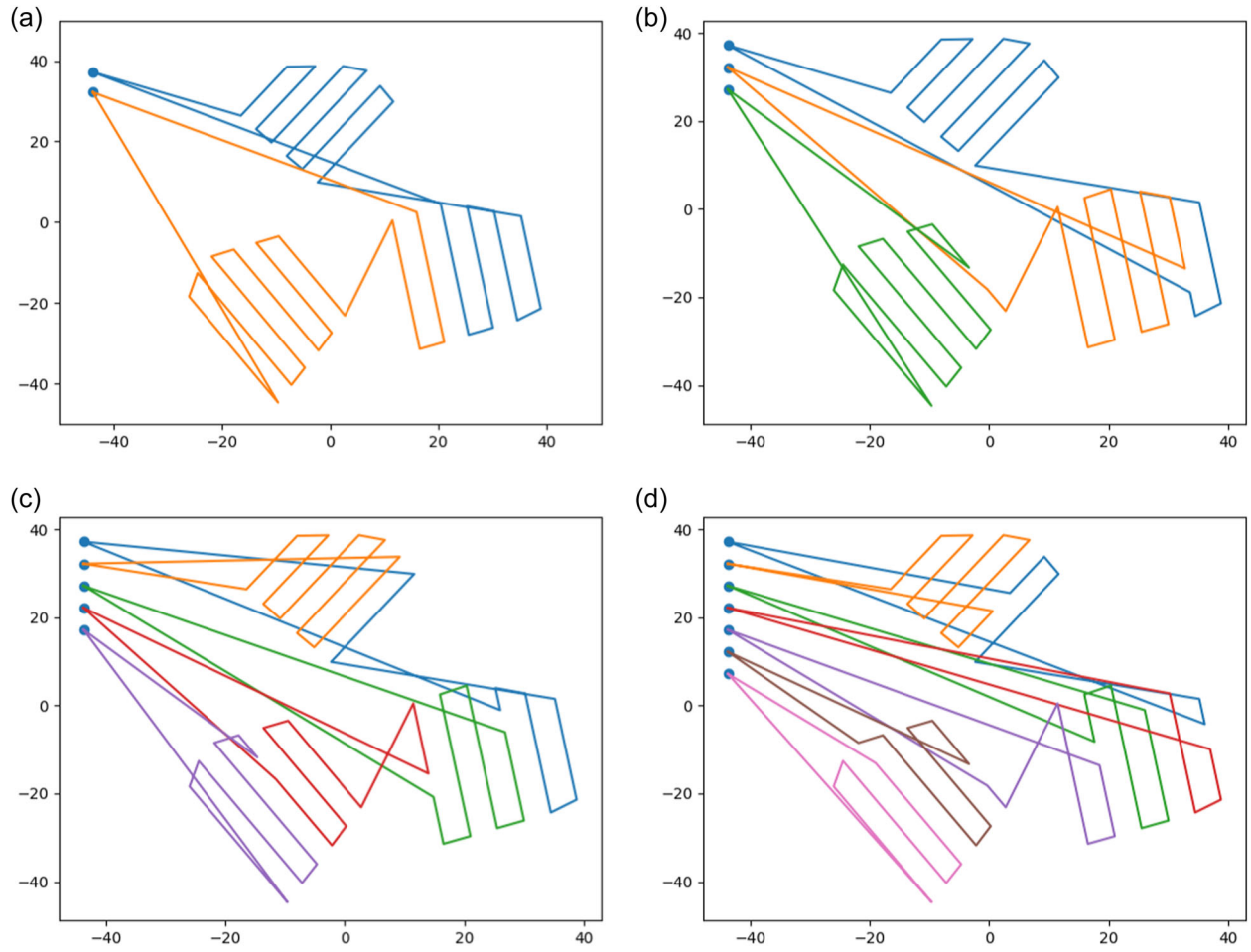


FIGURE 11 Algorithm performance results for three explored regions (a) two UAVs, (b) three UAVs, (c) five UAVs, and (d) seven UAVs.

TABLE 4 Mission cost and coefficient of variation (CV) results for different exploration areas and three UAVs.

Regions	Cost (m)	CV
2	289.86	2.77
3	537.58	4.89
4	658.86	2.30
5	655.85	1.08
6	732.95	3.56
7	840.76	1.28
8	849.50	3.90
9	1036.69	0.94
10	1130.46	0.72

TABLE 5 Mission parameters for CPU time experiments.

Parameter	Value
Area of each region	2500 m^2
Mission altitude	20 m
Lane spacing	10 m
Waypoint separation	10 m

5 | EXPERIMENTAL EVALUATION AND RESULTS

5.1 | Experiment's design and metrics

The proposed architecture has been validated using different kinds of tests in two main consecutive phases. A first version of the presented architecture was initially validated using the Flightmare simulator which did not included the use of Aerostack2 yet. Figure 10 shows example images generated during the experiments performed for three UAVs during an inspection task in a photovoltaic plant.

Then, a more elaborated version of the presented architecture was evaluated using three additional methodologies, including performance efficiency of the generated plans, simulation tests, and real flight tests. An important evaluation factor is the balance in mission distribution, measured through the coefficient of variation (CV) given by the following equation:

$$CV = \frac{SD}{Av} \times 100, \tag{12}$$

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

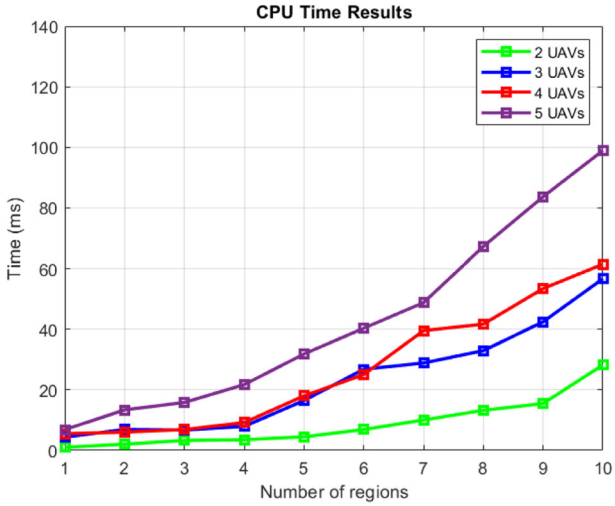


FIGURE 12 CPU time results for different UAVs and regions.

where *SD* is the standard deviation of the individual costs and *Av* is the average cost.

On the other hand, the mission cost is determined as the maximum distance or time it takes for one of the agents to complete the mission. Finally, the processing times in the planning and re-planning tasks are tabulated to estimate the system's response times. The simulation environment and the algorithms can be found at Computer Vision and Aerial Robotics Group Github account.[4]

5.2 | Performance efficiency analysis of the generated plans

Before evaluating the algorithms in the real and simulated environments, their performance was tested by analyzing quantitative measures of the generated plans. First, we evaluate the mission cost and the CV in multiple experiments using random generated regions and UAV initial positions. In this case, the measured mission cost is the maximum distance traveled by one agent. Results for multiple UAVs with three areas are presented in Table 3 and graphically in Figure 11. In addition, results for multiple areas and three UAVs are presented in Table 4.

In Tables 3 and 4 it can be noticed that the low *CV* in the UAV mission costs indicates a equitable distribution of the routes among the agents.

On the other hand, the CPU time of the algorithm will depend on the characteristics of the regions to be explored; for this reason, we set the parameters of each zone according to the values shown in Table 5.

Thus, we set the number of UAVs between 2 and 5 and the number of regions to explore between 1 and 10 using a laptop with Intel Core i7-7700HQ CPU, 3.8 GHz, and 16 Gb RAM running Ubuntu 22.04. The results are shown graphically in Figure 12.

Figure 12 show that the CPU time grows significantly as the number of UAVs increases; and this growth is moderate for a larger number of regions. However, in both cases this time remains relatively low. The optimization approach of BINPAT together with the heuristic solution for multiple area coverage perform this task faster when compared to other proposals in the literature.

For instance, in the research presented by (Xie & Chen, 2022) the authors use MATLAB on a MacBook Pro PC with a 2.6GHz 6-Core Intel Core i7 processor and 16 GB memory. Applying evolutionary algorithms they obtained CPU times between 183.3 ms for a travel distance of 1275.1 m and 265.3 ms for a travel distance of 2110.8 m. Their branch and bound technique reports 304.2 ms as minimum. In this case, the exact number of regions is not specified, but the authors state that the number of areas for these experiments was less than 10. They use between one and six UAVs.

On the other hand, the authors in (Vasquez-Gomez et al., 2018) perform their experiments with Matlab running on Intel Core i7 machine and 2 GB memory, obtaining 84 ms for five exploration areas and 89 ms for 10 regions. They use a single UAV.

In this case, we could not obtain the source code of the previously mentioned methods to run comparative experiments. However, compared to the first approach, our algorithm has lower CPU times for computers with similar processing features, obtaining a maximum of 98.92 ms for 10 regions and five UAVs. Compared to the second algorithm, ours has a lower CPU time if we consider the number of UAVs. In our tests, we obtained a time of 28.24 ms for two UAVs and 10 regions, but the authors of this study used a computer with a lower performance.

5.3 | Simulation results

We performed experiments with three to five UAVs in continuous flights, and flights with the need for re-planning with one and two aircraft unexpected events. These flights were performed with the help of virtual Iris quadcopters simulated with the Ignition Gazebo simulator. Table 6 shows the distance results for different number of zones and different UAVs in continuous flights.

Next, we performed experiments for three coverage areas, including the re-planning tasks for UAV unexpected events. In the flights with one lost aircraft, the remaining UAVs replan the routes to complete the mission. The results are shown in Table 7.

In Table 7, we exclude the lost UAV from *CV* calculations (bold values) in flights with one event. Additionally, during the flights with two events, the UAV1 was lost, then, when the UAV2 is lost, the UAV1 recovers itself again to accomplish the plan together with the rest of the agents; at this point, we exclude the results of UAV2 in the *CV* calculation (bold values), and keep those of UAV1. Tables 6 and 7 show the distance results of multiple UAVs when executing a mission. We also compute the *CV* to determine the degree of equality in the mission distribution. In continuous flights, as shown in Table 6,

[4]https://github.com/cvar-upm/coverage_path_planning

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 6 Simulation distance results for multiple UAVs and multiple exploration areas.

Zones	UAV1 (m)	UAV2 (m)	UAV3 (m)	UAV4 (m)	UAV5 (m)	Cost (m)	CV
2	80.63	81.41	76.65	–	–	81.41	3.21
	67.14	71.73	66.44	67.19	–	71.73	3.56
	61.65	62.66	54.2	60.27	57.36	62.66	5.82
3	110.52	107.86	89.74	–	–	110.52	11.01
	92.56	92.59	94.26	86.32	–	94.26	3.83
	78.8	81.92	77.08	76.15	82.11	82.11	3.45
4	138.79	141.52	144.5	–	–	144.5	2.02
	113.03	114.72	109.17	102.68	–	114.72	4.86
	96.08	97.19	92.55	98.77	97.25	98.77	2.43

TABLE 7 Simulation distance results for multiple UAVs in three exploration areas with replanning events.

Number of events	UAV1 (m)	UAV2 (m)	UAV3 (m)	UAV4 (m)	UAV5 (m)	Cost (m)	CV
1	11.53	161.44	164.95	–	–	164.95	1.52
	11.82	129.91	138.72	131.92	–	138.72	3.46
	11.97	115.44	118.93	105.62	108.71	118.93	5.43
2	152.74	32.16	164.78	–	–	164.78	5.36
	104.65	30.64	138.39	128.9	–	138.39	14.03
	107.81	34.83	115.3	119.49	130.06	130.06	7.86

CV remains low, assuming that the UAVs complete the mission without problems during the flight. In the plans in which replanning was necessary (Table 7), it can be observed how *CV* increases relatively little among the drones that have completed the mission; it also remains low even in the exceptional case in which an aircraft is recovered and can complete the flight plan with the rest of the agents.

To evaluate the CPU time in cases when the UAVs have to replan, we generate different flight plans for three zones, simulating one and two unexpected events during the flight to measure the time to take to the system to replan the tasks for the remaining UAVs. The results are shown in Table 8.

Table 8 show the low computational time of the proposed algorithms during planning and re-planning tasks. In this case, it can be observed that the planning times do not depend on the number of UAVs (as shown in Figure 12); this is because we perform the simulation tests with random area regions, which affects the CPU time. In the dynamic re-planning approach presented by (Hong et al., 2022) the authors use a a 2.6 GHz Intel i7 CPU and 16 GB RAM to perform their experiments, obtaining from 260 to 710 ms in each mission replanning while our method obtain a maximum of 14 ms in a CPU with similar features. In this context, our system maintains mission continuity by handling shorter replanning times. In addition, our algorithm exhibits an improved distribution of tasks among the agents (they report one test with three UAVs obtaining a CV of 14.03). Finally, graphical results are shown in Figures 13 and 14.

TABLE 8 Simulation CPU planning time results for replanning cases with multiple UAVs and three exploration areas.

Number of events	UAVs	CPU time (ms)			
		Planning	1st. re-planning	2nd. re-planning	Average
1	3	13.61	11.30	-	12.45
	4	16.37	12.46	-	14.42
	5	13.32	12.60	-	12.96
2	3	15.24	11.70	14.12	13.69
	4	13.51	14.91	12.62	13.68
	5	13.15	14.08	15.10	14.11

Figures 13 and 14 show the trajectories of the UAVs in a flight with two unexpected events. At the beginning of the operation, UAV1 returns to the launch point, and UAV2 takes the mission over together with UAV3. At a second time, UAV2 fails, and UAV1 recovers, so the system replans the mission for UAV1 and UAV3, taking over the paths for UAV2. Figure 14 also shows the altitude transitions to ensure the safety given by the Powell-BINPAT algorithm.

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

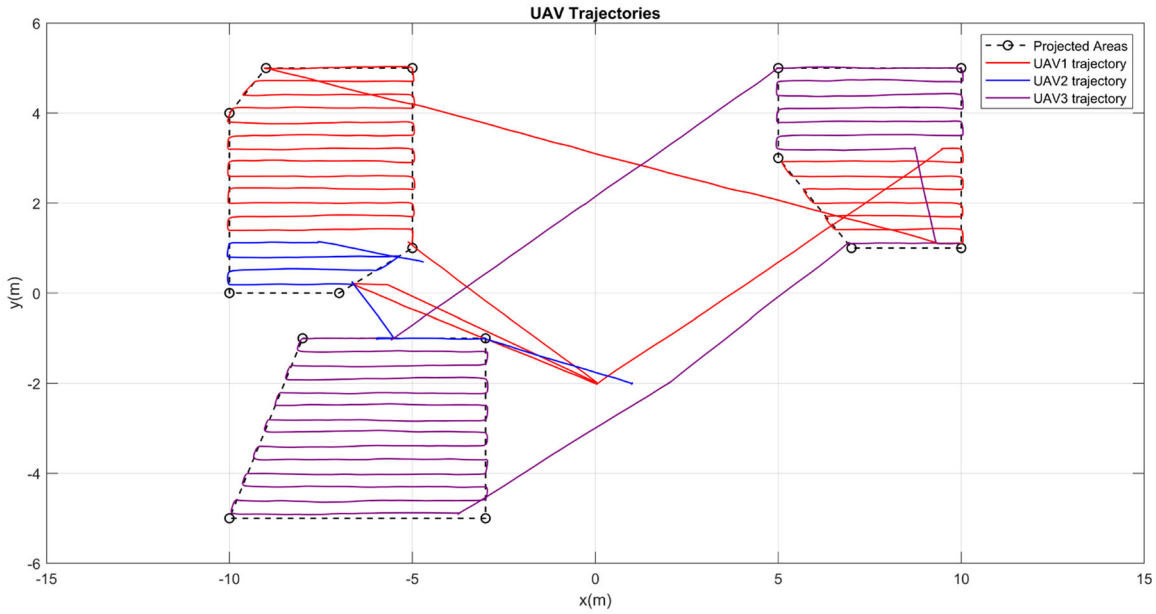


FIGURE 13 UAV trajectories top view.

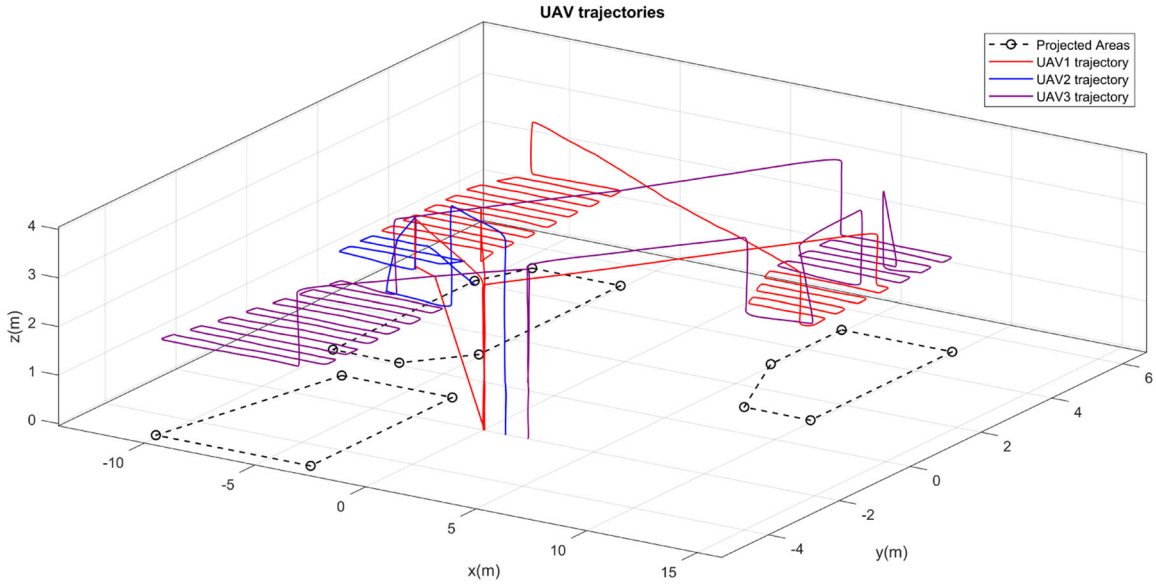


FIGURE 14 UAV trajectories lateral view.

5.4 | Real-flight results

Real flight tests were conducted with small and lightweight Bitcraze Crazyflie platforms[5] (dimensions: 92 × 92 × 29mm, weight: 27 g), as shown in Figure 15. Additionally, we use the Optitrack[6] motion capture system as localization method to perform precision indoor flights, avoiding distance measurement errors.

Regarding the experiments, we conducted three flight tests with three UAVs in three areas to corroborate the results obtained in the simulation stage. The first flight was continuous without re-planning tasks. For the second flight, we introduce an unrecoverable event in one of the UAVs during the execution of the mission. Finally, for the third flight, we initiate a return task in one of the UAVs; then, we introduce a failure event in a second UAV and recover the first UAV again to continue with the mission. Distance results are presented in Table 9. Besides, results for CPU times are presented in Table 10.

As the simulation environment (Section 5.3), the results for real-flight tests show that the algorithm distributes the tasks with a

[5]https://www.bitcraze.io/products/crazyflie-2-1/
[6]https://optitrack.com/

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

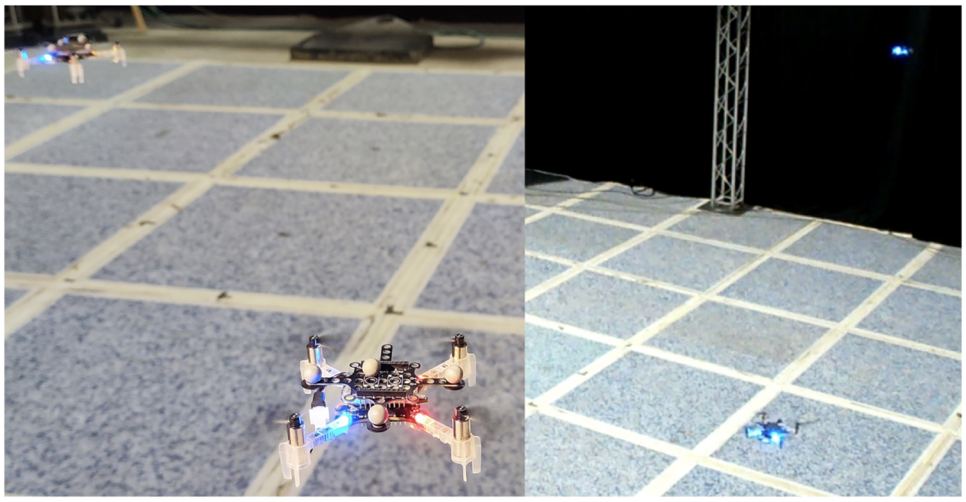


FIGURE 15 Aerial platforms used in the real flight tests (left) and part of the spatial area used for the experiments with real flights (right).

TABLE 9 Real flights distance results for three UAVs in three exploration areas and replanning events.

Flight number	UAV1 (m)	UAV2 (m)	UAV3 (m)	Cost	CV
0	30.29	31.22	27.07	31.22	7.37
1	44.46	44.83	7.62	44.83	0.58
2	41.83	17.87	42.92	42.92	1.83

TABLE 10 Real flight CPU time results for three UAVs in three exploration areas and replanning events.

Flight number	CPU time (ms)		
	Planning	1st. replanning	2nd. replanning
0	16.22	–	–
1	12.35	11.34	–
2	14.45	16.88	13.25

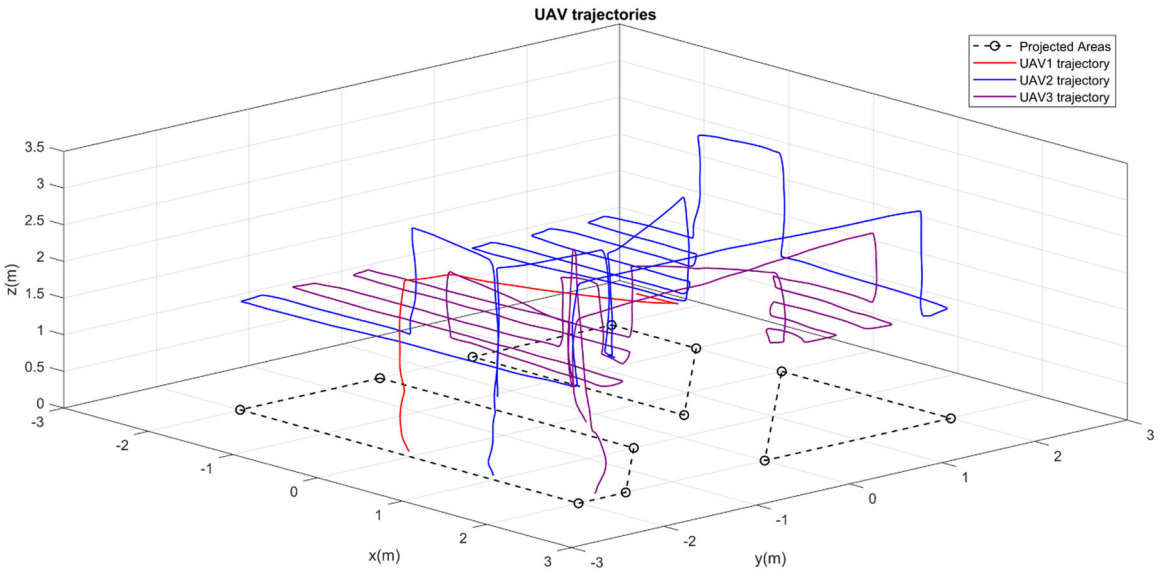


FIGURE 16 UAV trajectories in real flight with one replanning event.

relatively low CV; it is also possible to check again the efficiency in the computation times of the algorithm in the planning and replanning tasks. In addition, Figure 16, shows the UAV trajectories for a real flight experiment with one unexpected event.

Figure 16 shows trajectory data from a real flight in which UAV1 fails at the beginning of the mission, and UAV2 and UAV3 take their part of the mission flying at safety altitude during their respective transitions. A video showing

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

experiments with real flights can be found at CVAR-UPM Vimeo channel.[7]

6 | CONCLUSIONS AND FUTURE WORK

In summary, this work presents a solution for the development of a multi-UAV system for coverage tasks in separate areas with the feature of in-flight route re-planning. The results show that the system can distribute tasks equitably among the aircraft in an efficient way, even in the middle of the flight, when facing unexpected events. The results presented in Tables 3 and 4 show that our methods for multiple area CPP with a multi-UAV system are more efficient than others proposals in the state of the art; optimizing the cost of the mission adequately for a small number of areas. However, we cannot assure that the result will be cost-optimal in every case.

On the other hand, the strategies for early detection of errors and in-flight re-planning proposed for the mission execution system in section 4 has demonstrated their performances in simulation and real-world experiments in Section 5; enabling, therefore, the system to make changes and take the path-planning decisions. Also, the global monitoring system is efficient enough to perform the re-planning task during the flight in the middle of the mission.

For future works, we propose to improve the system autonomy by including collision detection algorithms that eliminate the need for safety altitudes. On the other hand, we consider developing swarm intelligence algorithms to manage missions during failures in complex behaviors and to compare them with the current proposal. Additionally, we propose to address the problem of planning and re-planning for heterogeneous multi-UAV systems.

ACKNOWLEDGMENTS

This work has been supported by the project COPILOT ref. Y2020/EMT6368 "Control, Monitoring and Operation of Photovoltaic Solar Power Plants by mean of synergic integration of Drones, IoT, and advanced communication technologies," funded by Madrid Government under the R&D Synergic Projects Program. We also acknowledge the support of the project "CORESENSE: A Hybrid Cognitive Architecture for Deep Understanding" Ref. No. 101070254 funded by European Union under the Horizon Europe Program. This work has also been supported by the project INSERTION ref. PID2021-127648OBC32, "UAV Perception, Control and Operation in Harsh Environments," funded by the Spanish Ministry of Science and Innovation under the program "Projects for Knowledge Generating." This work has been funded by the Community of Madrid through the project REF: IND2020/IND-17478 "Ayudas para la realización de Doctorados Industriales," especially regarding the work of the first author. The work of the fifth author is supported by the Spanish Ministry of Science and Innovation under its Program for Technical Assistants PTA2021-020671. We also would like to thanks to David Eugenio-Quiroga, Elisa Martinez-Abad and Alejandro Corpas-Calvo for their support in programming tasks.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Marco Andrés Luna http://orcid.org/0000-0003-1036-7538
Martin Molina http://orcid.org/0000-0001-7145-1974
Pedro Arias-Perez http://orcid.org/0000-0001-7166-9367
Pascual Campoy http://orcid.org/0000-0002-9894-2009

REFERENCES

Avellar, G.S., Pereira, G.A., Pimenta, L.C. & Iscold, P. (2015) Multi-uav routing for area coverage and remote sensing with minimum time. *Sensors*, 15(11), 27783–27803.

Balampanis, F., Maza, I. & Ollero, A. (2017) Spiral-like coverage path planning for multiple heterogeneous uas operating in coastal regions. In: *2017 International Conference on Unmanned Aircraft Systems (ICUAS), Miami, FL, USA*, IEEE, pp. 617–624.

Barrientos, A., Colorado, J., Cerro, J.d., Martinez, A., Rossi, C., Sanz, D. et al. (2011) Aerial remote sensing in agriculture: a practical approach to area coverage and path planning for fleets of mini aerial robots. *Journal of Field Robotics*, 28(5), 667–689.

Basiri, A., Mariani, V., Silano, G., Aatif, M., Iannelli, L. & Glielmo, L. (2022) A survey on the application of path-planning algorithms for multi-rotor uavs in precision agriculture. *The Journal of Navigation*, 75(2), 364–383.

Cabreira, T.M., Brisolara, L.B. & Paulo R.F.J. (2019) Survey on coverage path planning with unmanned aerial vehicles. *Drones*, 3(1), 4.

Chen, J., Qing, X., Ye, F., Xiao, K., You, K. & Sun, Q. (2022) Consensus-based bundle algorithm with local replanning for heterogeneous multi-uav system in the time-sensitive and dynamic environment. *The Journal of Supercomputing*, 78(2), 1712–1740.

Cho, S.W., Park, H.J., Lee, H., Shim, D.H. & Kim, S.-Y. (2021) Coverage path planning for multiple unmanned aerial vehicles in maritime search and rescue operations. *Computers & Industrial Engineering*, 161, 107612.

Choi, Y., Choi, Y., Briceno, S. & Mavris, D.N. (2020). Energy-constrained multi-uav coverage path planning for an aerial imagery mission using column generation. *Journal of Intelligent & Robotic Systems*, 97(1), 125–139.

Collins, L., Ghassemi, P., Esfahani, E.T., Doermann, D., Dantu, K. & Chowdhury, S. (2021). Scalable coverage path planning of multi-robot teams for monitoring non-convex areas. In: *2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China*, IEEE, pp. 7393–7399.

De Benedetti, M., D'Urso, F., Fortino, G., Messina, F., Pappalardo, G. & Santoro, C. (2017) A fault-tolerant self-organizing flocking approach for uav aerial survey. *Journal of Network and Computer Applications*, 96(2), 14–30.

Emel'yanov, S., Makarov, D., Panov, A.I. & Yakovlev, K. (2016) Multilayer cognitive architecture for uav control. *Cognitive Systems Research*, 39(4), 58–72.

Fernandez-Cortizas, M., Molina, M., Arias-Perez, P., Perez-Segui, R., Perez-Saura, D. & Campoy, P. (2023) Aerostack2: A software framework for developing multi-robot aerial systems. *arXiv preprint arXiv:2303.18237*.

[7]https://vimeo.com/847537551

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Flores Peña P., Luna, M.A., AleIsaac, M.S., Ragab, A.R., Elmenshawy, K., MartínGómez, D. et al. (2022) A proposed system for multi-uavs in remote sensing operations. *Sensors*, 22 (23), 9180.

Fourlas, G.K. & Karras, G.C. (2021) A survey on fault diagnosis and fault-tolerant control methods for unmanned aerial vehicles. *Machines*, 9(9), 197.

Gerkey, B.P. & Matarić, M.J. (2004) A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9), 939–954.

Hong, Y., Jung, S., Kim, S. & Cha, J. (2021) Autonomous mission of multi-uav for optimal area coverage. *Sensors*, 21(7), 2482.

Hong, Y., Jung, S., Kim, S. & Cha, J. (2022) Extensions of receding horizon task assignment for area coverage in dynamic environments. *IEEE Transactions on Aerospace and Electronic Systems*, 59(3), 3114–3128.

Hu, J., Niu, H., Carrasco, J., Lennox, B. & Arvin, F. (2022) Fault-tolerant cooperative navigation of networked uav swarms for forest fire monitoring. *Aerospace Science and Technology*, 123(2), 107494.

Huang, W.H. (2001) Optimal line-sweep-based decompositions for coverage algorithms. In: *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164)*, *Seoul*, *Korea (South)*, vol. 1. IEEE, pp. 27–32.

Jonker, R. & Volgenant, T. (1987) A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38, 325–340.

Khanam, Z., Saha, S., Ehsan, S., Stolkin, R. & Mcdonald-Maier, K. (2020) Coverage path planning techniques for inspection of disjoint regions with precedence provision. *IEEE Access*, 9, 5412–5427.

Kumar, K. & Kumar, N. (2023) Region coverage-aware path planning for unmanned aerial vehicles: a systematic review. *Physical Communication*, 59(7), 102073.

Li, J., Li, X. & Yu, L. (2018). Multi-uav cooperative coverage path planning in plateau and mountain environment. In: *2018 33rd Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, *Nanjing*, *China*, IEEE, pp. 820–824.

Luna, M.A., AleIsaac, M.S., Ragab, A.R., Campoy, P., Flores Peña, P. & Molina, M (2022). Fast multi-uav path planning for optimal area coverage in aerial sensing applications. *Sensors*, 22(6), 2297.

Luna, M.A., Isaac, M.S.A., Fernandez-Cortizas, M., Santos, C., Ragab, A.R., Molina, M. et al. (2023). Spiral coverage path planning for multi-uav photovoltaic panel inspection applications. In: *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*, *Warsaw*, *Poland*, IEEE, pp. 679–686.

Luna, M.A., Ragab, A.R., Isac, M.S.A., Peña, P.F. & Cervera, P.C. (2021) A new algorithm using hybrid uav swarm control system for fire-fighting dynamical task allocation. In: *2021 IEEE International Conference on Systems*, *Man*, *and Cybernetics (SMC)*, *Melbourne*, *Australia*, IEEE, pp. 655–660.

Macenski, S., Foote, T., Gerkey, B., Lalancette, C. & Woodall, W. (2022) Robot operating system 2: design, architecture, and uses in the wild. *Science Robotics*, 7 (66), eabm6074. https://doi.org/10.1126/scirobotics.abm6074

Mansouri, S.S., Kanellakis, C., Fresk, E., Kominiak, D. & Nikolakopoulos, G. (2018) Cooperative coverage path planning for visual inspection. *Control Engineering Practice*, 74, 118–131.

Maza, I. & Ollero, A. (2007) Multiple uav cooperative searching operation using polygon area decomposition and efficient coverage algorithms. In: Alami, R., Chatila, R., & Asama, H. (eds) *Distributed Autonomous Robotic Systems 6*. Tokio: Springer, pp. 221–230.

Melo, A.G., Pinto, M.F., Marcato, A.L., Honório, L.M. & Coelho, F.O. (2021) Dynamic optimization and heuristics based online coverage path planning in 3d environment for uavs. *Sensors*, 21(4), 1108.

Meng, B.-b., Gao, X. & Wang, Y. (2009) Multi-mission path re-planning for multiple unmanned aerial vehicles based on unexpected events. In: *2009 International Conference on Intelligent Human-Machine Systems and Cybernetics*, *Hangzhou*, *China*, Vol. 1. IEEE, pp. 423–426.

Meng, L., Qing, S. & Jun, Z.Q. (2017). Uav path re-planning based on improved bidirectional rrt algorithm in dynamic environment. In: *2017 3rd International Conference on Control*, *Automation and Robotics (ICCAR)*, *Nagoya*, *Japan*, IEEE, pp. 658–661.

Molina, M., Camporredondo, A., Bavle, H., Rodriguez-Ramos, A. & Campoy, P. (2019) An execution control method for the aerostack aerial robotics framework. *Frontiers of Information Technology & Electronic Engineering*, 20(1), 60–75.

Mou, Z., Zhang, Y., Gao, F., Wang, H., Zhang, T. & Han, Z. (2021) Deep reinforcement learning based three-dimensional area coverage with uav swarm. *IEEE Journal on Selected Areas in Communications*, 39 (10), 3160–3176.

Nedjati, A., Izbirak, G., Vizvari, B. & Arkat, J. (2016) Complete coverage path planning for a multi-uav response system in post-earthquake assessment. *Robotics*, 5(4), 26.

Pinto, M.F., Honório, L.M., Marcato, A.L., Dantas, M.A., Melo, A.G., Capretz, M. et al. (2021) Arcog: an aerial robotics cognitive architecture. *Robotica*, 39(3), 483–502.

Powell, M.J. (1964) An efficient method for finding the minimum of a function of several variables without calculating derivatives. *The computer journal*, 7(2), 155–162.

Sanchez-Lopez, J.L., Pestana, J., De La Puente, P. & Campoy, P. (2016). A reliable open-source system architecture for the fast designing and prototyping of autonomous multi-uav systems: simulation and experimentation. *Journal of Intelligent & Robotic Systems*, 84(1), 779–797.

Semsch, E., Jakob, M., Pavlicek, D. & Pechoucek, M. (2009) Autonomous uav surveillance in complex urban environments. In: *2009 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology*, *Milan*, *Italy*, Vol. 2. IEEE, pp 82–85.

Shang, Z., Bradley, J. & Shen, Z. (2020) A co-optimal coverage path planning method for aerial scanning of complex structures. *Expert Systems with Applications*, 158, 113535.

Tahir, A., Böling, J., Haghbayan, M.-H. & Plosila, J. (2020). Development of a fault-tolerant control system for a swarm of drones. In: *2020 International Symposium ELMAR*, *Zadar*, *Croatia*. IEEE, pp. 79–82.

Torres, M., Pelta, D.A., Verdegay, J.L. & Torres, J.C. (2016) Coverage path planning with unmanned aerial vehicles for 3d terrain reconstruction. *Expert Systems with Applications*, 55, 441–451.

Valente, J., DelCerro, J., Barrientos, A. & Sanz, D. (2013) Aerial coverage optimization in precision agriculture management: A musical harmony inspired approach. *Computers and electronics in agriculture*, 99, 153–159.

Vasquez-Gomez, J.I., Herrera-Lozada, J. -C. & Olguin-Carbajal, M. (2018) Coverage path planning for surveying disjoint areas. In: *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*, *Dallas*, *TX*, *USA*, IEEE, pp. 899–904.

Vazquez-Carmona, E.V., Vasquez-Gomez, J.I., Herrera-Lozada, J.C. & Antonio-Cruz, M. (2022) Coverage path planning for spraying drones. *Computers & Industrial Engineering*, 168, 108125.

Wang, H., Chen, M. & Fu, P. (2021) A distributed fault-tolerant mechanism for mission-oriented unmanned aerial vehicle swarms. *International Journal of Communication Systems*, 34(8), e4789.

Xie, J. & Chen, J. (2022) Multiregional coverage path planning for multiple energy constrained UAVs. *IEEE Transactions on Intelligent Transportation Systems*, 23(10), 17366–17381.

Xie, J., Garcia-Carrillo, L.R. & Jin, L. (2018) An integrated traveling salesman and coverage path planning problem for unmanned aircraft systems. *IEEE Control Systems Letters*, 3(1), 67–72.

Yu, X., Jin, S., Shi, D., Li, L., Kang, Y. & Zou, J. (2020) Balanced multi-region coverage path planning for unmanned aerial vehicles. In: *2020 IEEE International Conference on Systems*, *Man*, *and Cybernetics (SMC)*, *Toronto*, *ON*, *Canada*, IEEE, pp. 3499–3506.

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Zhou, Y., Rao, B. & Wang, W. (2020) Uav swarm intelligence: recent advances and future trends. *IEEE Access*, 8, 183856–183878.

Zhu, K., Han, B. & Zhang, T. (2021) Multi-uav distributed collaborative coverage for target search using heuristic strategy. *Guidance, Navigation and Control*, 1(1), 2150002.

Ziquan, Y., Zhang, Y., Jiang, B., Jun, F. & Ying, J. (2022) A review on fault-tolerant cooperative control of multiple unmanned aerial vehicles. *Chinese Journal of Aeronautics*, 35(1), 1–18.

How to cite this article: Luna, M. A., Molina, M., Da-Silva-Gomez, R., Melero-Deza, J., Arias-Perez, P. & Campoy, P. (2024) A multi-UAV system for coverage path planning applications with in-flight re-planning capabilities. *Journal of Field Robotics*, 41, 1480–1497. https://doi.org/10.1002/rob.22342

15564967, 2024, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.22342 by Cochrane Chile, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License